

Проект по СПО

Паралелно пресмятане на числото на Ойлер (e) чрез конвейерна архитектура

Гаврил Христов

ФН: 6MI0800460

спец. Компютърни Науки
курс 3, втори поток, група 7

2026

Съдържание

1	Увод	2
1.1	Развитие на числото e в ред на Тейлър	2
1.2	Цел на проекта	2
2	Анализ на алгоритъма	2
2.1	Метод за паралелно пресмятане на факториели и суми	2
2.2	Метод за балансиране на задачите	2
2.3	Сложност на аритметиката с произволна точност	3
2.4	Последователност на изпълнение на алгоритъма	3
2.5	Функционален анализ	4
2.6	Нефункционален анализ (Технологичен анализ)	5
3	Проектиране и реализация	5
3.1	Функционално проектиране	5
3.1.1	Описание на функциите и коментар на алгоритмите	5
3.1.2	Диаграми	6
3.1.3	Потребителски интерфейси и екрани на реакция	7
3.1.4	Ръководство за потребителя (Параметри)	8
3.2	Технологично проектиране	8
4	Тестване	8
4.1	Методология на тестване	8
4.2	Резултати	9
4.3	Анализ и обобщение	10
5	Източници	11

1 Увод

1.1 Развитие на числото e в ред на Тейлър

Числото e притежава уникалното свойство да бъде основа на естествената експоненциална функция. Математическата дефиниция чрез ред на Тейлър е:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + \frac{1}{1!} + \frac{1}{2!} + \dots$$

Този ред осигурява бърза сходимост, тъй като факториелът в знаменателя нараства свръхекспоненциално.

1.2 Цел на проекта

Основната цел на проекта е проектирането и реализацията на паралелна система за изчисляване на числото e с висока точност. Системата използва конвейерна архитектура за разпределяне на изчисленията, с цел постигане на оптимална скалируемост чрез балансирано натоварване на нишките спрямо сложността на изчисление на факториелите.

2 Анализ на алгоритъма

2.1 Метод за паралелно пресмятане на факториели и суми

Предизвикателството при паралелното пресмятане на редове от типа на Тейлър се състои в последователната зависимост на факториелите. В проекта използвам декомпозиция на домейна, при която изчислителният обхват $[0, N]$ се разделя на интервали. Паралелизмът се реализира чрез конвейерна обработка, при която всяка нишка предава междинния си резултат на следващата, след като е завършила пресмятането на своя интервал. Този подход минимизира конфликтите при достъп до паметта [2].

2.2 Метод за балансиране на задачите

За да осигурия равномерно натоварване на нишките в конвейерната архитектура, ще разгледаме тези два основни подхода: предварителна оценка на изчислителния обхват и адаптивно разпределение на задачите спрямо сложността им.

Оценка на обхвата чрез апроксимация на Стирлинг

Апроксимацията на Стирлинг е фундаментална за проекта, тъй като позволява предварително определяне на броя членове N в реда на Тейлър, необходими за постигане на желаната точност от p десетични знака. За постигане на точност от p десетични знака е необходимо да определим броя членове N така, че остатъкът $R_N \approx \frac{1}{(N+1)!}$ да бъде по-малък от 10^{-p} . За да бъде остатъкът по-малък от 10^{-p} , прилагаме логаритмична трансформация върху знаменателя:

$$\ln((N+1)!) > p \cdot \ln(10)$$

Използвайки формулата на Стирлинг $\ln(n!) \approx n \ln n - n$, неравенството се трансформира до:

$$(N+1) \ln(N+1) - (N+1) \approx p \ln(10)$$

При големи стойности на N , пренебрегвайки линейните членове спрямо логаритмичните, получаваме:

$$N \ln N \approx p \ln 10$$

Чрез преминаване към десетичен логаритъм и допълнителни алгебрични преобразувания, достигаем до практическата формула за планиране на работния диапазон:

$$N \approx \frac{p}{\log_{10} p}$$

Тъй като изчисляването на факториел с голяма стойност е изчислително много скъпо, тази оценка е критична за дефиниране на минимално необходимата горна граница на изчисленията.

Динамична декомпозиция с тежести

Тъй като сложността на изчисление на факториелите в реда на Тейлър расте нелинейно с всеки следващ член, линейното разделяне на обхвата би довело до сериозен дисбаланс — нишките, обработващи интервали с по-високи индекси, биха били значително по-натоварени.

За да се компенсира този ефект, използвах следния метод за разпределение, базиран на намаляващи тежести. Изчислителният обхват $[0, N]$ се разделя на $C = T \times I$ части (където T е броят нишки, а I – грануларността). На всяка част i се присвоява тежест W_i , която намалява линейно с нарастването на индекса:

$$W_i = C - i \quad \text{за } i \in [0, C - 1]$$

Общият размер на всеки интервал се определя пропорционално на неговата тежест спрямо общата сума на тежестите:

$$\text{Размер}_i = \max \left(1, \left\lfloor \frac{W_i}{\sum W} \cdot N \right\rfloor \right)$$

Този подход гарантира, че началните интервали (където факториелите са малки и се изчисляват бързо) съдържат повече членове, докато крайните интервали (които са изчислително много "скъпи") обхващат по-малък брой членове. Допълнително, чрез параметъра за грануларност I , разпределението може да се настройва фино, което допълнително оптимизира балансирането на натоварването и минимизира времето за престой (idle time) на нишките в конвейера.

2.3 Сложност на аритметиката с произволна точност

При изчисляване на математически константи с хиляди или милиони десетични знаци, основното изчислително време се измества от самото обхождане на цикъла към сложността на аритметичните операции. С нарастването на индекса в реда на Тейлър, локалните числители и знаменатели достигат огромни размери [4].

В конвейерния модел, междинните пресмятания в отделните етапи използват целочислена аритметика. Въпреки че съвременното умножение на големи числа е алгоритмично оптимизирано, финалното деление на акумулирания числител и знаменател изисква алгоритми за произволна точност. Забавянето при тази последна стъпка може да се превърне в тясно място (bottleneck), ако паралелизмът не е съобразен с нарастващата цена на самите аритметични операции.

2.4 Последователност на изпълнение на алгоритъма

За да се проследи цялостният жизнен цикъл на изчислителния процес, изпълнението на алгоритъма може да бъде представено в строга хронологична последователност.

Процесът стартира с фазата на подготовка, при която се изчислява общият брой необходими членове на реда на Тейлър чрез апроксимацията на Стирлинг. Веднага след това

се прилага алгоритъмът за декомпозиция с намаляващи тежести, който разделя изчислителния домейн на оптимално балансирани интервали.

Втората стъпка е инициализацията на конвейерната структура. Създават се комуникационните канали (опашки), свързващи отделните етапи. Работните нишки се инстанцират и на всяка се присвоява циклично подмножество от подготвените интервали за обработка.

Следва същинската стартова фаза. Главният процес поставя базовата начална стойност в първата опашка на конвейера и подава сигнал за едновременно стартиране на всички работни процеси (нишки).

По време на самото изпълнение, всяка нишка пресмята локално частичния числител и знаменател за текущия си интервал. Когато приключи, тя блокира и изчаква получаването на натрупаната сума от предходния етап през съответната входна опашка. След получаването на данните, нишката ги комбинира с локалните си резултати чрез бърза целочислена аритметика и предава новата стойност нататък по веригата.

Алгоритъмът приключва с финализираща фаза. Когато междинните данни достигнат до последния етап на конвейера, се извършва реалното деление с произволна точност на вече акумулираните огромни числа. Генерира се финалното десетично представяне на константата e , което се връща към главния процес за запис в изходния файл.

2.5 Функционален анализ

Проектът реализира високопрецизно пресмятане на математическата константа e чрез развитието ѝ в ред на Тейлър. Тъй като пресмятането изисква работа с огромни числа, паралелният модел трябва да адресира не само разпределението на задачите, но и комуникационните разходи между нишките. За избор на оптимална архитектура са анализирани следните литературни образци с аналогична функционалност:

Метод на двоично разделяне (Binary Splitting)

Изследванията в [1] и [3] разглеждат метода на двоично разделяне като стандарт за пресмятане на редове от рационални числа с висока точност. Алгоритъмът използва декомпозиция от тип „разделяй и владей“ (Divide and Conquer), изграждайки рекурсивно дърво от частични суми. Този подход балансира големината на операндите и силно оптимизира аритметичните операции, но изисква дървовидна синхронизация на данните на всяко ниво от рекурсията, което усложнява паралелния контрол при ограничен брой нишки. Този метод е по-подходящ за архитектури с голям брой процесори.

Балансиран конвейерен паралелизъм (Load-balanced Pipeline)

В [2] се анализира изграждането на паралелни конвейери с балансирано натоварване. Там описват потокова архитектура, при която изчислителният домейн се разделя на етапи, свързани чрез буферирани опашки за предаване на съобщения. Според изследването, за да се избегне изчакването на конвейера, е ключово разпределението на тежестта да бъде съобразено със сложността на всеки етап, което вдъхновява използваната в моя проект статична декомпозиция с тежести.

Оценка на елементарни функции с произволна точност

[4] се фокусира върху същинския изчислителен проблем при работа с големи числа. Авторът доказва, че при хиляди и милиони итерации, времето за изпълнение е доминирано от огромните умножения и деления в паметта, а не от самия цикъл. Това налага използването на локални целочислени оптимизации (натрупване на локален знаменател) преди финалното скъпо деление, подход, имплементиран като защитен механизъм в линейния конвейер на моя проект.

Таблица 1: Сравнителен анализ на паралелните архитектури

Образец	Максима- лен паралели- зъм	Синхрон- ност	Тип декомпо- зиция	Баланси- ране	Гранулар- ност	Контекс- тни парамет- ри
Моят проект	Ограничен от бр. интервали	Локално- синхронен	Линеен конвейер	Статично	Променли- ва	Брой членове и Десетична точност
Binary Splitting [1, 3]	Много висок ($O(N)$ в листата)	Синхронен	Дървовид- на	Статично	Фина	Дълбочина на рекурсията
Load- Balanced Pipeline [2]	Среден (според дължина- та)	Асинхро- нен	Потокова	Динамич- но / Статично	Средна до Едра	Брой етапи в конвейера

2.6 Нефункционален анализ (Технологичен анализ)

При *метода на двоично разделяне* [1, 3] традиционно се използват езици от ниско ниво (като C/C++) в комбинация с високооптимизирани библиотеки за аритметика с произволна точност (напр. GMP - GNU Multiple Precision). Изпълнението се реализира върху мултипроцесорни платформи със споделена памет (Shared Memory), като синхронизацията по нивата на рекурсивното дърво изисква строг контрол чрез бариери и общи променливи.

При *конвейерните архитектури* [2] обикновено се залага на модели с предаване на съобщения (Message Passing) или буфери в споделената памет. Тези архитектури често се реализират чрез пулове от нишки (Thread Pools) в Java или C++, като фокусът е върху непрекъснатия поток на данните.

3 Проектиране и реализация

3.1 Функционално проектиране

3.1.1 Описание на функциите и коментар на алгоритмите

Имплементацията на проекта е направена на езика Python. Проекта е разделен на няколко основни логически модула (функции и класове), които реализират паралелното изчисление в конкретните файлове на проекта:

- **Спомагателни математически функции (helper.py):**

Функцията `estimate_terms(precision)` изчислява нужния брой членове на реда чрез формулата на Стирлинг спрямо желаната точност. Функцията `distribute_work(terms, threads, interval)` осъществява декомпозицията. Тя прилага алгоритъма с намаляващи тежести, за да върне списък от кортежи (интервали), които балансират изчислителната тежест.

- **Управление на конвейера (main.py):**

Функцията `setup()` инициализира конфигурацията: задава контекста за прецизност (чрез `decimal`), генерира масив от `Queue` обекти (по една за всяка граница на интервал)

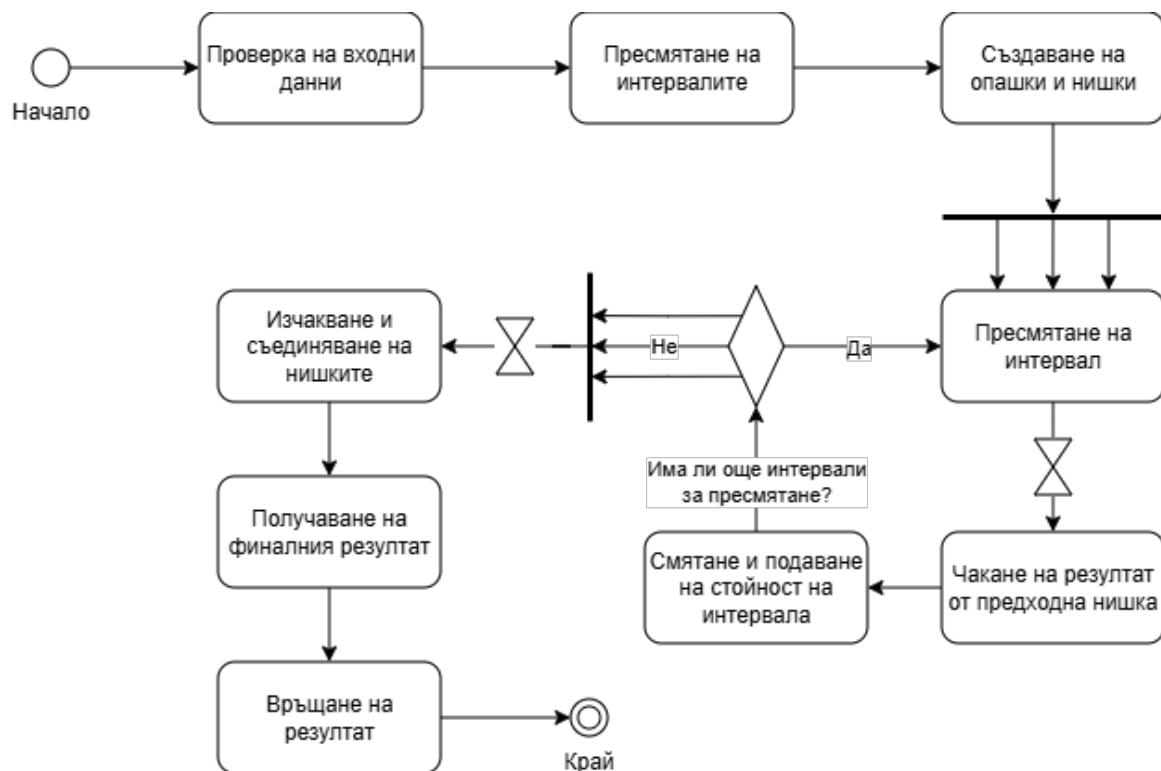
и инстанцира обектите `Worker`. Функцията `run(workers, queues)` оркестрира конвейера, като поставя базовата начална стойност (1.0) в първата опашка, стартира всички нишки (`start()`), изчаква тяхното приключване (`join()`) и извлича крайния резултат от последната опашка.

- **Работни нишки и синхронизация (клас `Worker` в `worker.py`):**

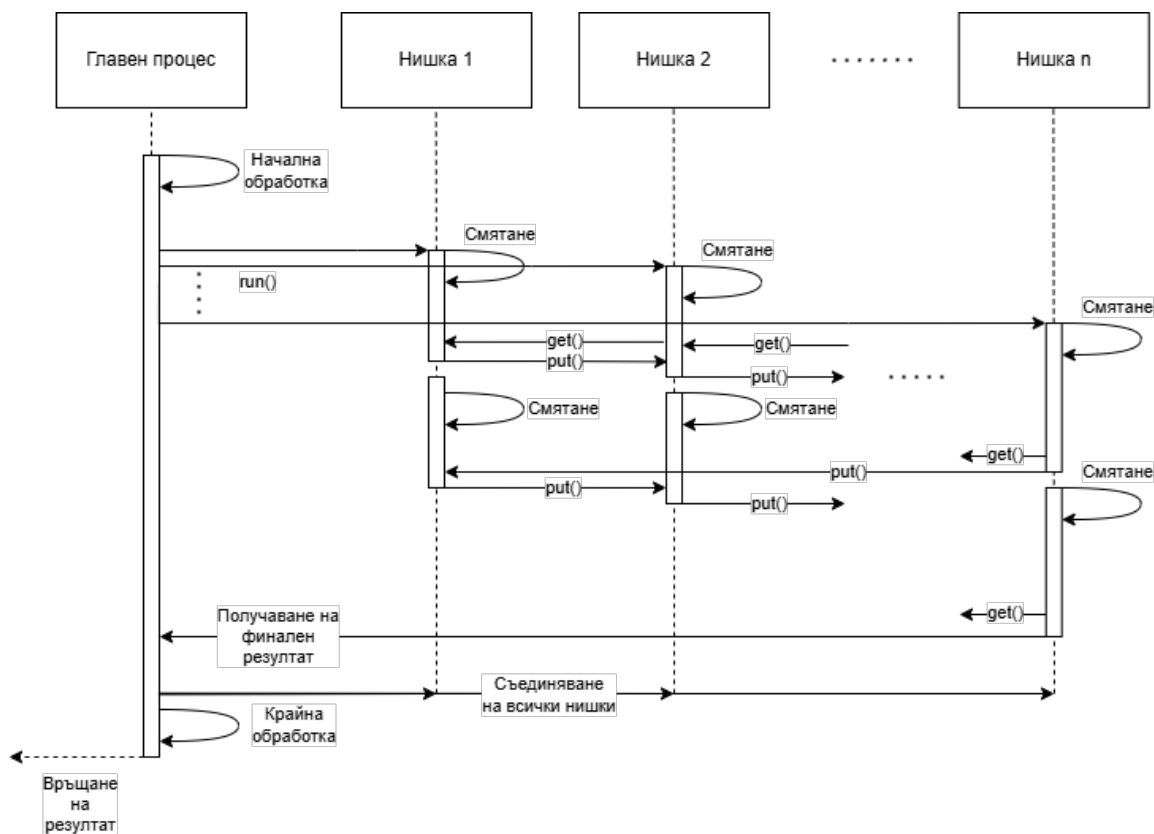
Всяка нишка изпълнява предефиниран `run()` метод. Алгоритъмът вътре обхожда назначените интервали циклично (чрез стъпка `threads_count`), което позволява на една нишка да участва в няколко етапа на конвейера. За текущия интервал се изчисляват локални числител и знаменател. Критичната синхронизация се случва, когато нишката извика `get()` върху съответната входна опашка, блокирайки до получаване на предходната сума. Резултатът се комбинира чрез целочислена аритметика и се изпраща (`put()`) в следващата опашка. При достигане на последния интервал, алгоритъмът завършва с реално деление на големи числа и връща крайното десетично число e .

3.1.2 Диаграми

Следните UML диаграми представят процесите в проекта:



Фигура 1: UML Activity diagram



Фигура 2: UML Sequence diagram

3.1.3 Потребителски интерфейси и екрани на реакция

Тъй като приложението е конзолно (CLI), интерфейсите "екрани" представляват текстови команди, а реакцията на системата — форматиран изход в терминала.

Интерфейсен екран:

Потребителят подава заявка към системата чрез терминала. Пример за стартиране с 4 нишки, точност 1000 знака и грануларност 2:

```
> python main.py -p 1000 -t 4 -i 2 result.txt
```

Екран от реакцията на приложението:

Системата извежда детайлна информация за конфигурацията на конвейера, разпределението на интервалите и времето за изпълнение:

```
[INFO] Calculating e to 1000 decimal places.
[INFO] Taylor series terms: 333
[INFO] Using 4 worker(s) in a linear pipeline.
[INFO] Intervals per thread: 2 (8 stages total, ~41.6 terms/stage)
[INFO] Work intervals: (0, 74), (74, 138), (138, 193), (193, 239), (239, 276), ... (321, 333)
Setup time:          232600
Calculation time:    1845300
Total time:          2321100
Result written to: result.txt
```

Пресметнат резултат в result.txt:

```
2.7182818284590452353602874713526624977572470936999595749669676277240
76630353547594571382178525166427427466391932003059921817413596629043
57290033429526059563073813232862794349076323382988075319525101901157
38341879307021540891499348841675092447614606680822648001684774118537
42345442437107539077744992069551702761838606261331384583000752044933
82656029760673711320070932870912744374704723069697720931014169283681
90255151086574637721112523897844250569536967707854499699679468644549
05987931636889230098793127736178215424999229576351482208269895193668
03318252886939849646510582093923982948879332036250944311730123819706
84161403970198376793206832823764648042953118023287825098194558153017
56717361332069811250705603135875160016131847177835837621661546712606
38418179159281323571613203575519282122660522466683909395375715018336
33429982080524326655651863422552513867200022010837951130127723603105
63420198383746737632295110692566012730419086290181501426096564993831
4236108474525189050475806519053471982322251659465314645190
```

3.1.4 Ръководство за потребителя (Параметри)

Системата изисква инсталирана версия на Python 3.14 (Free-Threading/NoGIL), която е необходима за постигане на истинско паралелно изпълнение.

Синтаксис и аргументи:

- `-t`, `--threads` (опционален): Брой нишки (по подразбиране 1).
- `-p`, `--precision` (задължителен): Брой десетични знаци.
- `-i`, `--interval` (опционален): Коефициент на грануларност.
- `-q`, `--quiet` (флаг): Потиска информационния изход.
- `file` (позиционен аргумент): Път до изходния файл.

3.2 Технологично проектиране

За нуждите на разработката, тестването и анализа на скалируемостта е използвана следната конфигурация:

4 Тестване

4.1 Методология на тестване

За оценка на производителността и скалируемостта на реализираната система е разработен тестов план, базиран на следните ключови параметри, използвани за анализ на паралелното изпълнение:

- n (**точност**): Брой десетични знаци, до които се пресмята числото e . Този параметър директно определя сложността на изчислението и броя на членовете от реда на Тейлър.
- g (**грануларност**): Коефициент на сегментиране (интервали на нишка), който дефинира фиността на разпределението на изчислителните задачи.

Категория	Параметър	Стойност
CPU	Модел	Intel Core i7-13700K
	Архитектура	x86_64
	Общо ядра	16 (8 Р-ядра, 8 Е-ядра)
	Нишки	24
	Честота	3.4 GHz (Base) / 5.4 GHz (Turbo)
	Кеш (L1i/L1d/L2/L3)	768 KB / 640 KB / 24 MB / 30 MB
RAM	Размер	64 GB
OS	Версия	Windows 11

- $T_p(i)$: Измерено време за изпълнение на конкретен опит (тест) с номер i при използване на зададен брой нишки p .
- $T_p = \min(T_p(i))$: Най-доброто (минимално) постигнато време измежду всички проведени опити за конкретна конфигурация. Тази стойност служи като база за изчисляване на показателите за производителност.
- $S_p = T_1/T_p$: Ускорение, показващо съотношението между времето за последователно изпълнение (T_1) и най-доброто време за паралелно изпълнение с p нишки (T_p).
- $E_p = S_p/p$: Ефективност, представяща колко ефективно се използват наетите ресурси (нишки) в сравнение с идеалното ускорение.

4.2 Резултати

p	Tp(1)	Tp(2)	Tp(3)	Tp	Sp	Ep
1	304124500.0	286595800.0	286825800.0	286595800.0	1.0	1.0
2	160151300.0	159005600.0	159714200.0	159005600.0	1.8024258265117707	0.90121291325
4	99087700.0	96975700.0	97356700.0	96975700.0	2.9553362337162814	0.73883405842
6	85634500.0	85934400.0	85409800.0	85409800.0	3.355537654929528	0.55925627582
8	83724200.0	83632200.0	82578400.0	82578400.0	3.4705903723007467	0.43382379653
10	82492700.0	83000800.0	82391500.0	82391500.0	3.478463190984507	0.34784631909
12	82583900.0	83888400.0	83255300.0	82583900.0	3.4703592346692274	0.28919660288
14	90003200.0	90385500.0	89325100.0	89325100.0	3.2084576451635654	0.22917554608
16	90115100.0	90173800.0	89595000.0	89595000.0	3.19879234332273	0.19992452145
18	99403900.0	98222500.0	98216700.0	98216700.0	2.917994597660072	0.16211081098
20	110863200.0	102286600.0	100277600.0	100277600.0	2.8580241250289196	0.14290120625
22	107233000.0	106254000.0	105701300.0	105701300.0	2.7113744107215334	0.12324429139
24	112156600.0	113003000.0	111581000.0	111581000.0	2.568500013443149	0.10702083389

4.3 Анализ и обобщение

[Тук следва да бъде направен анализ на получените резултати, коментар относно скалируемостта на програмата и нейните ограничения при различна натовареност.]

5 Източници

- [1] B. Haible, & T. Papanikolaou (1997). *Fast multiprecision evaluation of series of rational numbers*. <https://www.ginac.de/CLN/binsplit.pdf>
- [2] M. Kamruzzaman, S. Swanson, & D. M. Tullsen (2013). *Load-balanced pipeline parallelism*. <https://scispace.com/pdf/load-balanced-pipeline-parallelism-3ad4huixe2.pdf>
- [3] H. Vestermark (2025). *Exploring the Binary Splitting Method*. <https://www.hvks.com/Numerical/Downloads/HVE%20Exploring%20Binary%20Splitting%20Method.pdf>
- [4] R. P. Brent (1976). *Fast Multiple-Precision Evaluation of Elementary Functions*. <https://scispace.com/pdf/fast-multiple-precision-evaluation-of-elementary-functions-4ml8gpg10p.pdf>